SWIFT (Secure World Infrastructure Frontier Triage)

1. Executive Overview

SWIFT is an autonomous AI-driven cybersecurity platform designed to detect, triage, and remediate vulnerabilities at scale. It addresses the AI-driven security gap created by frontier models capable of discovering and exploiting vulnerabilities faster than human defenders.

2. Architecture Overview

SWIFT Architecture: Input → Control Plane → Sandbox → Remediation → Evidence → Human Approval Control Plane: - MCP Gateway (tool control) - Permission Enforcement (built) - Audit Logging (next) - Triage Engine (future) Sandbox: - Docker-based isolated execution - Safe vulnerability reproduction Remediation: - Patch generation - Validation testing Evidence: - Full report bundles for compliance

3. Completed Work

Environment: - Python, Docker, security tools installed Codebase: - config/settings.py (central config system) - server/permissions.py (security enforcement layer) Capabilities: - Read-only execution enforced - Tool access restricted - Policy-based validation implemented

4. Key Engineering Learnings

- Separation of config vs configs is critical - Circular imports can break system initialization - Security systems must default to deny - Every tool execution must be controlled and logged

5. Security Model

SWIFT enforces: - Read-only default permissions - Allowlisted tool execution - No direct system access by AI - Mandatory sandboxing for all execution - Human approval before remediation This ensures enterprise-grade safety and compliance.

6. Code Insight

config/settings.py: - Centralizes all runtime configuration - Loads environment variables - Defines policy constants server/permissions.py: - Validates all tool requests - Enforces allowlists - Blocks unsafe operations

7. Roadmap

Next Steps: 1. server/forensic_logger.py (audit logging) 2. MCP tool wrappers 3. Triage engine 4. Evidence bundle generator 5. Remediation engine

8. Claude Code Transition Notes

- Use strict tool interfaces - Never bypass permission layer - Always log every action - Maintain sandbox isolation - Validate outputs before execution